Princess Sumaya University for Technology
جامعة الأميرة سميّة للتكنولوجيا

Performance Evaluation of CPU Scheduling Algorithms

Prepared by:

Lana Ismail, 20210229, lan20210229@std.psut.edu.jo

Basma Shawkat, 20200854, BAS20200854@std.psut.edu.jo

Supervised by:

Dr. Osama Al-Haj Hassan

Second semester / 2023 - 2024

Table of contents

Introduction..... 3

Methodology (Algorithms)..... 4

- First-Come First-Served (FCFS) – Non-Preemptive 4
- Shortest-Job-First (SJF) – Non-Preemptive 6

Results..... 8

Conclusions..... 12

References..... 13

Introduction

In operating systems, efficient CPU scheduling stands as a cornerstone for optimizing system performance and resource utilization. The allocation of CPU time between competing processes is a critical task that affects factors such as response time, throughput, and overall system efficiency. In this report, we delve into evaluating two basic CPU scheduling algorithms: First-Come First-Served (FCFS) and Shortest-Job-First (SJF), which are both non-preemptive.

Our main goal in this report is to compare the performance of FCFS and SJF algorithms. We want to learn more about their operational characteristics, strengths, and limitations, which will lead to a better understanding of CPU scheduling processes.

We aim to gain a better understanding of how FCFS and SJF perform in different situations by carefully executing and testing them. By examining metrics such as average turnaround time and average waiting time, we try to determine which algorithms perform best in different workload conditions.

This report thoroughly examines CPU scheduling algorithms and covers both theoretical concepts and practical applications. By analyzing the performance of FCFS and SJF, we aim to provide valuable insights into the complex world of operating systems design and optimization.

In the following sections, we will go over the methods used to implement the FCFS and SJF algorithms, present our experimental results, and discuss the significance of our findings. Through this project, we aim to provide a detailed view of the importance of CPU scheduling in modern computing systems, allowing for better decision-making and system development.

Methodology (Algorithms)

First algorithm: First-Come First-Served (FCFS) – Non-Preemptive

The First-Come First-Served (FCFS) algorithm is one of the simplest CPU scheduling algorithms used by operating systems to manage processes. It uses a FIFO queue in its implementation, which stands for First In First Out.

FCFS scheduling is non-preemptive which means, once a process is assigned to the CPU, it keeps the CPU busy until it completes its job. [2]

As its name implies, the FCFS algorithm schedules processes based on the order of their arrival in the ready queue. First, it places the arriving processes/jobs at the end of the ready queue, and the process that requests the CPU first gets the allocation first. Then, when any process enters the FIFO queue, its Process Control Block (PCB) gets linked to the end of the queue. Finally, when the CPU becomes free, the first process in the queue gets allocated to it and the currently running process gets removed from the queue. [3][4]

Advantages of FCFS Scheduling: [1][3][4]

- It is easy to implement.
- It is suitable for simple systems and scenarios where computational overhead needs to be minimized.
- It has no context switch between the processes, which makes it better for the processes with comparatively large burst time.
- It is well suited for batch systems where the longer time periods for each process are often acceptable.

Disadvantages of FCFS Scheduling: [1][4]

- It has a high Waiting Time compared to other scheduling algorithms.
- It suffers from the Convoy effect, which happens when short processes are forced to wait behind long processes.
- Not very efficient compared to its simplicity.
- It is not suitable for time-sharing operating systems.
- Since it is non-preemptive, it does not release the CPU until it completes its current job execution completely.

Pseudocode: [5]

```
queue = empty queue
clock = 0

while (processes not finished)
    if (new process arrives at clock time)
        add the process to the end of the queue

    if (CPU is idle and queue is not empty)
        remove the first process from the queue
        set the start time of the process to the current clock time
        run the process for its burst time
        set the finish time of the process to the current clock time

increment the clock by 1
```

Second algorithm: Shortest-Job-First (SJF) – Non-Preemptive

In its non-preemptive variation , the shortest job first (SJF) algorithm executes the shortest job first by prioritizing tasks according to their burst time. The scheduler runs the process in ascending order after classifying them according to burst time. This can improve system throughput and reduce average waiting time. But if shorter processes keep coming in, longer ones might expire. SJF is effective at reducing waiting times, although it does require advanced knowledge of burst time. [6]

Advantages of shortest job first (SJF): [8]

- Minimize average waiting time for processes.
- Enhances system throughput by prioritizing shorter jobs.
- Simple and easy to implement.
- Can lead to efficient resource utilization when burst times are accurately known.

Disadvantages of shortest job first(SJF): [6][7]

- Longer processes may expire if shorter ones continuously arrive.
- Requires knowing burst time in advance, which may not always be feasible.
- May not be suitable for real-time systems or environments with unpredictable workload variations.
- May results in poor performance if burst time estimations are inaccurate.

Pseudocode:

```
current_time = 0

process_queue = empty

completed_processes = 0

shortest_index = -1

while there are remaining processes to execute:

    shortest_index = -1

for each process in the queue:

        if process has arrived and (shortest_index == -1 or its burst time is shorter than the

current shortest):

            shortest_index = index of this process

 if shortest_index is still -1:  // No process has arrived yet

        while next process arrival time > current_time:

            increment current_time

 else:  // Process with the shortest burst time has been found

        wait until current_time reaches the arrival time of the selected process

        execute the selected process until completion

        update current_time to reflect the completion time of the process

        increment completed_processes

        remove the completed process from the queue


calculate average waiting time and turnaround time if needed
```

Results

Arrival time (AT) − is the time when the process enters the ready queue.

Burst time (BT) − is the time it takes for a process to complete its execution.

Completion time (CT) − is the time at which the process has been terminated.

Turn-around time (TAT) − is the overall time from arrival to completion. TAT is written as:

Turn-around time (TAT) = Completion time (CT) – Arrival time (AT)

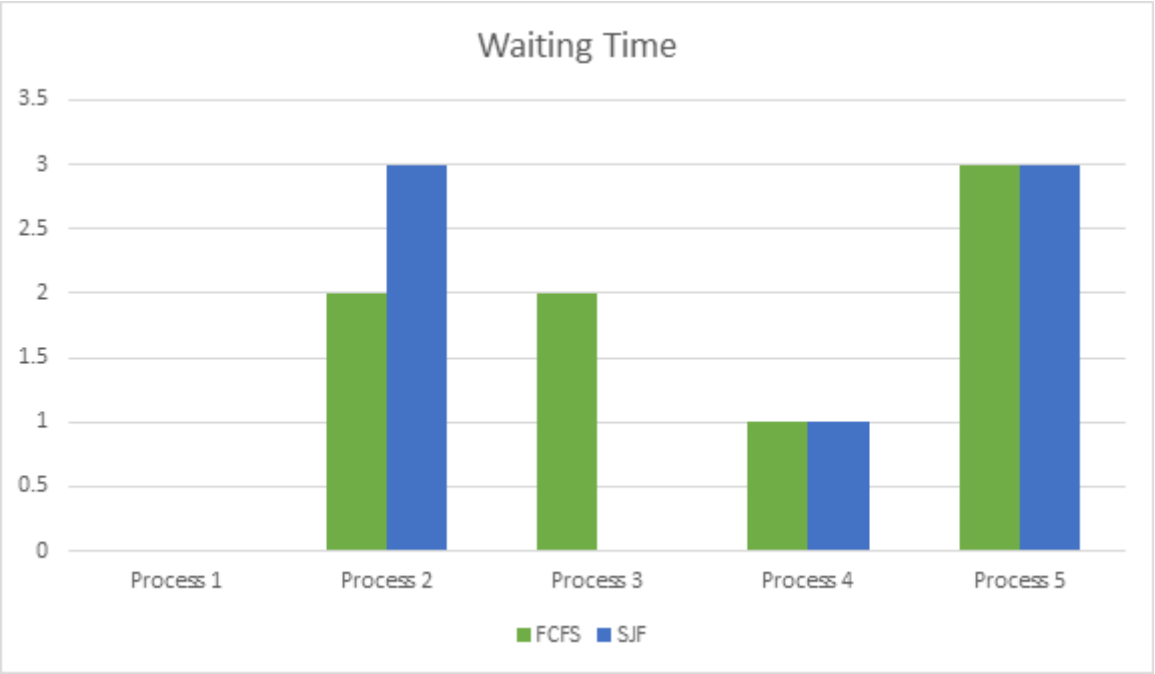
or, TAT = Burst time (BT) + Waiting time (WT)

Waiting time (WT) − is the amount of time a process spends waiting for its allocation while the previous process is in the CPU for execution. WT is written as:

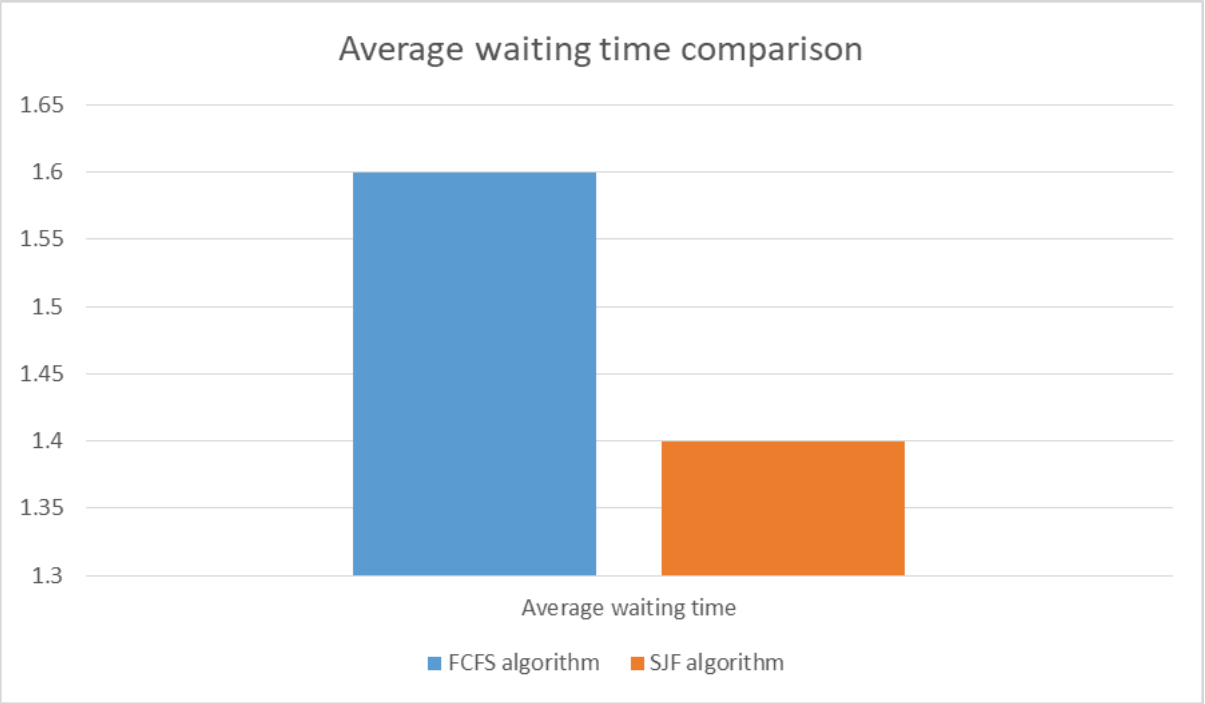
Waiting time (WT) = Turn-around time (TAT) – Burst time (BT) [1]

Waiting time comparison:

Process No.	Arrival Time	Burst Time	Waiting Time for FCFS	Waiting Time for SJF
1	0	4	0	0
2	2	2	2	3
3	4	1	2	0
4	6	3	1	1
5	7	5	3	3

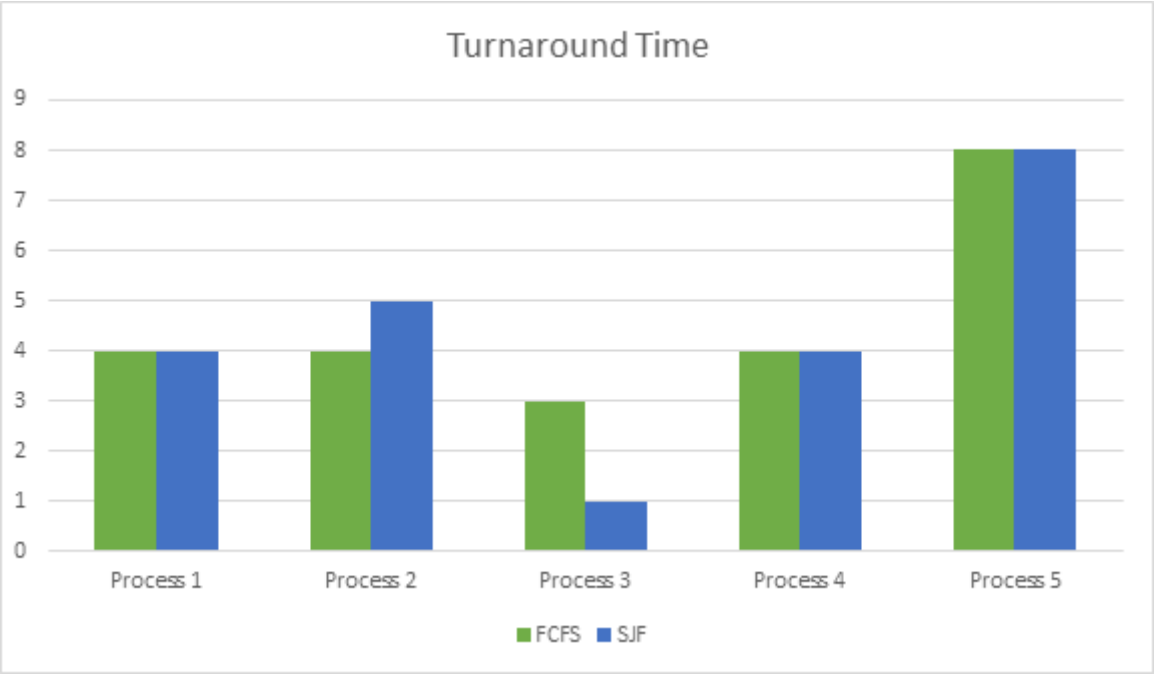


	FCFS algorithm	SJF algorithm
Average waiting time	1.6 ms	1.4 ms

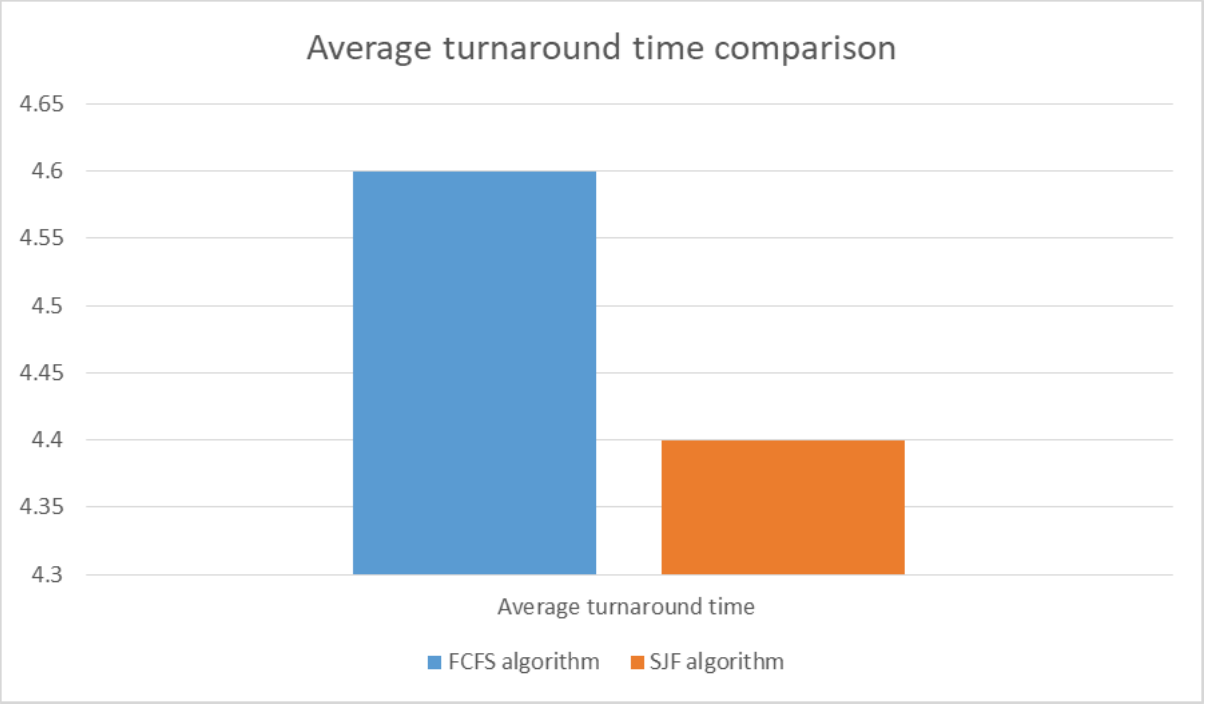


Turnaround Time comparison:

Process No.	Arrival Time	Burst Time	Turnaround Time for FCFS	Turnaround Time for SJF
1	0	4	4	4
2	2	2	4	5
3	4	1	3	1
4	6	3	4	4
5	7	5	8	8



	FCFS algorithm	SJF algorithm
Average turnaround time	4.6 ms	4.4 ms



The Gantt chart for FCFS:

P1	P2	P3	P4	P5

0 4 6 7 10 15

The Gantt chart for SJF:

P1	P3	P2	P4	P5

0 4 5 7 10 15

Conclusions

In conclusion, this report has explored the fundamental CPU scheduling algorithms, First-Come First-Served (FCFS) and Shortest-Job-First (SJF) to evaluate their performance in optimizing system efficiency. By comparing their operational characteristics, strengths, and limitations, we gained valuable insights into their practical applications.

FCFS, despite its simplicity and ease of implementation, can lead to higher average waiting times, especially when longer jobs arrive earlier in the queue. This is because FCFS processes tasks in the order they arrive, without considering the length of their burst times. If a longer job arrives before shorter ones, all subsequent shorter jobs will have to wait until the longer job is completed, leading to increased waiting times for those shorter jobs.

On the other hand, SJF tends to result in lower average waiting times. SJF prioritizes shorter jobs, executing them before longer ones. This scheduling method reduces waiting times for shorter processes, as they are less likely to be delayed by longer processes. [7]

In our scenario, SJF exhibited lower average waiting and turnaround times compared to FCFS. Specifically, SJF showed an average waiting time of 1.4 ms and an average turnaround time of 4.4 ms, while FCFS had respective metrics of 1.6 ms for average waiting time and 4.6 ms for average turnaround time. This highlights the efficiency of SJF in optimizing system resource utilization and enhancing overall performance.

In summary, while SJF generally performs better in terms of average waiting and turnaround times, its effectiveness depends on the burst times and arrival order of processes. FCFS, on the other hand, is simpler but may result in longer waiting times. Both algorithms have their advantages and disadvantages, and the choice between them depends on the specific requirements and characteristics of the system. [6]

References

[1] Chakraborty, Arnab. “FCFS Scheduling.” *Tutorialspoint*, 5 April 2023, https://www.tutorialspoint.com/fcfs-scheduling. Accessed 13 May 2024.

[2] “FCFS Scheduling Algorithms in OS (Operating System) - javatpoint.” *Javatpoint*, https://www.javatpoint.com/os-fcfs-scheduling. Accessed 12 May 2024.

[3] “FCFS Scheduling Full Form | Know The Full Form of FCFS Scheduling - BYJU'S.” *BYJU'S*, https://byjus.com/gate/fcfs-scheduling-full-form/. Accessed 12 May 2024.

[4] Jain, Sandeep. “First Come First Serve – CPU Scheduling (Non-Preemptive).” *GeeksforGeeks*, 3 November 2023, https://www.geeksforgeeks.org/first-come-first-serve-cpu-scheduling-non-preemptive/. Accessed 8 May 2024.

[5] Khandelwal, Neetika. “First Come, First Serve ñ CPU Scheduling | (Non-preemptive).” *Tutorialspoint*, 22 August 2023, https://www.tutorialspoint.com/first-come-first-serve-n-cpu-scheduling-non-preemptive. Accessed 7 May 2024.

[6] “Shortest Job First (SJF) Scheduling in Operating System.” *DataFlair*, https://data-flair.training/blogs/shortest-job-first-sjf-scheduling-in-operating-system/. Accessed 15 May 2024.

[7] Jain, Sandeep. “Difference between FCFS and SJF CPU scheduling algorithms.” *GeeksforGeeks*, 8 August 2022, https://www.geeksforgeeks.org/difference-between-fcfs-and-sjf-cpu-scheduling-algorithms/. Accessed 15 May 2024.

[8] Jain, Sandeep. “Advantages and Disadvantages of various CPU scheduling algorithms.” *GeeksforGeeks*, 12 May 2023, https://www.geeksforgeeks.org/advantages-and-disadvantages-of-various-cpu-scheduling-algorithms. Accessed 15 May 2024.